

GPT-5.5 Direct-Edit Quality Frontier, 50 Runs Per Cell

What changed when the preliminary 20-run result was extended to 50 valid runs per topology

By Adam Owada, with Codex
July 1, 2026

Abstract

This paper reports the 50-run-per-cell extension of the GPT-5.5 direct-edit quality frontier experiment on RuleLedger v3. The experiment compares four staged GPT-5.5 subagent topologies: high root with high leaves, xhigh root with high leaves, high root with xhigh leaves, and xhigh root with xhigh leaves. Each cell now contains 50 valid scored runs, for 200 total measured implementations.

The preliminary 20-run result made high root plus high GPT-5.5 leaves look like a clear winner. The 50-run result keeps that cell in first place on observed mean quality and token efficiency, but the evidence is more nuanced. High root plus high leaves scored the best mean quality, 0.7495, with a 95% bootstrap confidence interval of [0.7021, 0.7962]. Xhigh root plus xhigh leaves was second at 0.7308, xhigh root plus high leaves was third at 0.7190, and high root plus xhigh leaves was fourth at 0.6885.

The larger sample therefore changes the conclusion from "high/high is clearly best" to "high/high is the best observed default, but not decisively separated from the other top cells." The high/high advantage over xhigh/xhigh is only +0.0187 mean quality with a bootstrap interval crossing zero. Its advantage over xhigh/high is +0.0305, also unresolved. Its advantage over high/xhigh is larger at +0.0610 and is close to resolving, but still has a lower confidence bound just below zero.

The more durable result is about cost and reasoning interactions. Xhigh root reasoning did not provide a reliable average-quality lift. Xhigh leaves were not a general improvement and were actively unattractive when paired with a high root. Xhigh/xhigh remained the best upper-tail configuration, with the best top-quartile mean, but it spent about 35% more implementation tokens than high/high. In practice, high/high is still the recommended default if the goal is mean quality per token, while xhigh/xhigh is the configuration to study when the goal is peak-run hunting.

Decision Summary

Developer goal	Recommended configuration	Evidence
Best observed mean quality	High root + high GPT-5.5 leaves	Mean quality 0.7495, highest among the four cells
Best token efficiency	High root + high GPT-5.5 leaves	1.538e-07 quality per GPT-5.5 implementation token, best in the matrix
Best upper-tail behavior	Xhigh root + xhigh GPT-5.5 leaves	Best top-quartile mean, 0.9756, and 5 of the top 10 runs
Avoided default	High root + xhigh GPT-5.5 leaves	Lowest mean quality, 0.6885, despite higher leaf token spend
Main statistical caution	Do not over-rank the top three cells	Pairwise intervals among GQF0, GQF1, and GQF3 overlap zero
Practical next experiment	Reduce coordination load or make integration stricter	Six GPT-5.5 leaves add expensive search; the root still has to compress it into one coherent semantics model

Experiment Matrix

All runs used the same frozen RuleLedger v3 starter project, hidden test suite, scoring profile, prompt templates, and staged direct-edit workflow. The judge was GPT-5.5 xhigh for every run.

Cell	Root reasoning	Leaf reasoning	Leaf count	Leaf model	Write mode	Valid runs
GQF0	high	high	6	GPT-5.5	direct edit	50
GQF1	xhigh	high	6	GPT-5.5	direct edit	50
GQF2	high	xhigh	6	GPT-5.5	direct edit	50
GQF3	xhigh	xhigh	6	GPT-5.5	direct edit	50

The staged topology was:

```
Frozen RuleLedger v3 starter
-> GPT-5.5 root planning stage
-> six GPT-5.5 direct-edit leaf stages
-> GPT-5.5 root integration stage
-> public tests, hidden tests, scoring, judge review, usage parsing
```

The six leaves retained the prior role split: TypeScript parsing and compatibility; TypeScript replay, reporting, and performance; Python parsing and compatibility; Python replay, reporting, and performance; cross-language parity and regression review; and adversarial review.

Execution Audit

The 50-run result pools two validated batches:

Batch	Directory	Runs per cell	Notes
Original frontier	runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6	20	Initial 80-run experiment
Extension	runs/20260630T061938-gpt55_direct_quality_frontier_50-gpt55_frontier_50_r21_r50_j7_j6	30	Rerun-repaired 120-run extension

The extension initially produced usage-limit artifacts: GQF2 r35-r50 and all GQF3 r21-r50 had invalid implementation failures, and every extension judge phase failed from the same usage ceiling. Those zero scores were not treated as signal. The invalid phases were rerun with a shorter workspace path and lower concurrency. All 120 extension implementations completed, all 120 judges completed, and the final validation pass succeeded.

The final Stage 11 validation status for the extension is `passed`. The pooled analysis script also passed its own checks: exactly 50 rows per cell, exactly 20 original and 30 extension rows per cell, no duplicate run IDs, complete artifacts, no preserved failure phases, no usage warnings, and no non-positive quality scores.

The reproducible analysis command is:

```
python scripts\analyze_gpt55_frontier_50.py
```

It writes derived analysis outputs under `runs/analysis/gpt55_direct_quality_frontier_50`.

Primary Results

Cell	Configuration	Runs	Mean quality	95% bootstrap CI	Median	Top quartile	Hidden correctness	Judge	GPT-5.5 impl tokens	Quality/token
GQF0	High root + high leaves	50	0.7495	[0.7021, 0.7962]	0.7808	0.9607	0.7227	0.7350	5.029M	1.538e-07
GQF1	Xhigh root + high leaves	50	0.7190	[0.6775, 0.7636]	0.6892	0.9669	0.6974	0.7252	6.213M	1.200e-07
GQF2	High root + xhigh leaves	50	0.6885	[0.6450, 0.7335]	0.6457	0.9142	0.6660	0.6516	5.735M	1.222e-07
GQF3	Xhigh root + xhigh leaves	50	0.7308	[0.6847, 0.7791]	0.6873	0.9756	0.7188	0.7188	6.791M	1.105e-07

High/high remains the best observed mean performer. It also has the best median, the best mean hidden-test score, the best performance score, the lowest mean implementation token count, and the best quality-per-token score.

The story is not a clean monotonic reasoning ladder. Raising only the leaves from high to xhigh hurt the high-root cell. Raising only the root from high to xhigh also did not help the high-leaf cell. The all-xhigh cell recovered enough to place second by mean and first by upper tail, but at the highest token cost.

Original 20 vs Extension 30

Cell	Original 20 quality	Extension 30 quality	Pooled 50 quality	Interpretation
GQF0	0.7693	0.7363	0.7495	Original lead shrank
GQF1	0.7047	0.7286	0.7190	Original underperformance moderated
GQF2	0.6845	0.6912	0.6885	Stable low-ranking cell
GQF3	0.7321	0.7299	0.7308	Very stable mean

This split explains why the result felt counter-intuitive. The first 20 runs were directionally useful, but the extension changed the spacing. GQF0 stayed on top, yet its lead over GQF1 and GQF3 became much smaller. GQF3’s mean was almost unchanged; GQF1 improved; GQF2 stayed weak.

Pairwise Quality Comparisons

Comparison	Mean delta	95% bootstrap CI	Probability left > right
GQF0 minus GQF1	+0.0305	[−0.0338, +0.0928]	0.8230
GQF0 minus GQF2	+0.0610	[−0.0038, +0.1247]	0.9677
GQF0 minus GQF3	+0.0187	[−0.0478, +0.0846]	0.7152
GQF1 minus GQF2	+0.0305	[−0.0316, +0.0940]	0.8353
GQF1 minus GQF3	−0.0118	[−0.0747, +0.0532]	0.3623
GQF2 minus GQF3	−0.0423	[−0.1071, +0.0211]	0.0974

At 50 runs per cell, the pairwise result supports three tiers rather than a single decisive ranking:

- 1. GQF0 is the best observed mean and the best practical default.
- 2. GQF1 and GQF3 are close enough to GQF0 that their exact order remains uncertain.
- 3. GQF2 is the weakest observed cell, especially relative to GQF0 and GQF3.

The GQF0-over-GQF2 comparison is close to resolving; its lower bootstrap bound is only −0.0038. The GQF0-over-GQF1 and GQF0-over-GQF3 comparisons are not close to decisive.

Reasoning Effects

Effect	Mean quality delta	95% bootstrap CI	Probability positive
Xhigh root minus high root, pooled	+0.0059	[−0.0396, +0.0510]	0.6007
Xhigh leaves minus high leaves, pooled	−0.0246	[−0.0695, +0.0210]	0.1459
Xhigh root minus high root with high leaves	−0.0305	[−0.0934, +0.0338]	0.1714
Xhigh root minus high root with xhigh leaves	+0.0423	[−0.0218, +0.1068]	0.8986
Xhigh leaves minus high leaves with high root	−0.0610	[−0.1252, +0.0028]	0.0316
Xhigh leaves minus high leaves with xhigh root	+0.0118	[−0.0515, +0.0753]	0.6425

The clearest interaction is that xhigh leaves are not a free upgrade. With a high root, xhigh leaves reduced mean quality by 0.0610 and added about 0.706M implementation tokens. With an xhigh root, xhigh leaves had a small positive observed effect, but the interval remains wide.

One plausible interpretation is coordination load. Xhigh leaves may produce more ambitious and internally reasoned local edits, but the root still has to integrate six branches into one coherent TypeScript/Python semantics model. If the root is not also spending enough reasoning effort, xhigh leaf output can increase integration burden more than it increases usable solution quality. Even with an xhigh root, the quality lift is uncertain and token cost is high.

Upper Tail and Peak Runs

The best single run in the 50-run pool was `GQF0_direct_r32`, with quality `0.9904`. That is a change from the original 20-run paper, where the best single run came from `GQF3`. However, the upper tail still favors `GQF3`:

Cell	Top-quartile mean	Best run	Top-10 runs represented
GQF0	0.9607	0.9904	3
GQF1	0.9669	0.9861	1
GQF2	0.9142	0.9788	1
GQF3	0.9756	0.9893	5

This split matters operationally. If a workflow can afford many expensive attempts and select the best result after hidden or high-fidelity evaluation, `xhigh/xhigh` may be worth testing further. If the workflow wants the best expected result from a single run, `high/high` remains the better default.

Is 50 Runs Per Cell Enough?

Fifty runs per cell is enough to reject the most naive version of the original story: the top cells are not separated by large margins. It is also enough to show that `high root plus xhigh leaves` is not an attractive default in this task. It is not enough to prove the exact order of `GQF0`, `GQF1`, and `GQF3`.

The reason is simple variance. The pooled quality standard deviations are between about `0.157` and `0.171`. With `n=50`, the standard error of a cell mean is still roughly `0.022` to `0.024`. The observed top-cell gaps are `0.0187` to `0.0305`, which are the same size as one standard error. A `0.06` gap is borderline at this sample size; a `0.10` gap would be much more convincing.

A rough two-sample power calculation with quality standard deviation near `0.165` implies:

True mean gap	Approximate runs per cell for 80% power
0.10	about 43
0.06	about 119
0.05	about 171
0.03	about 475

That does not mean every future experiment needs hundreds of runs. It means that 50 runs per cell can support practical ranking decisions when gaps are large, but cannot make tiny differences look decisive. For this experiment, the right confidence statement is:

High/high is the best observed and most token-efficient default. `GQF3` is the best upper-tail configuration. The exact mean-quality ranking among `GQF0`, `GQF1`, and `GQF3` remains uncertain at 50 runs per cell.

Practical Interpretation

The 50-run extension strengthens the case for conservative reasoning budgets in multi-agent coding topologies. More reasoning did not monotonically increase mean quality. Instead, the topology behaved like a coordination system:

- High leaves gave the root smaller, cheaper material to integrate.
- `Xhigh` leaves increased leaf-side token spend and did not reliably improve final integrated quality.
- `Xhigh` root effort was not enough by itself to raise the high-leaf cell.
- `Xhigh` root plus `xhigh` leaves created the strongest upper tail, but not the strongest average or efficiency.

The result is especially relevant because all cells used GPT-5.5 leaves in direct-edit mode. This was already the higher-quality branch suggested by the Spark experiments. The remaining question was not "proposal or direct edit?" but "how much reasoning effort should the root and leaves spend once direct editing is chosen?" On RuleLedger v3, the answer is not simply "as much as possible."

Recommendations

For future RuleLedger-style subagent work:

1. Use high root plus high GPT-5.5 direct-edit leaves as the default quality-per-token setting.

2. Keep xhigh/xhigh as a peak-run or best-of-N candidate, not as the default single-run setting.
3. Do not run high root plus xhigh leaves unless testing a specific hypothesis about integration support.
4. Test whether fewer GPT-5.5 leaves improve mean quality by reducing root integration burden.
5. Add stricter root integration constraints or a post-integration consistency pass before spending more reasoning on leaves.
6. Treat 20-run cells as exploratory and 50-run cells as directional unless observed gaps are around 0.10 or larger.

Limitations

This experiment is a controlled benchmark result, not a universal law about GPT-5.5. RuleLedger v3 rewards global semantic consistency across TypeScript and Python implementations; other tasks may reward broader search more. All leaves used the same six-role structure, so the result does not isolate whether different role designs would benefit more from xhigh reasoning. The judge was held constant at GPT-5.5 xhigh, so judge behavior is not part of the matrix.

The confidence intervals are bootstrap intervals over observed runs, not a guarantee that future harness versions, model snapshots, or prompt changes will preserve the same ordering. The extension also required a usage-limit repair workflow. The final data are valid after rerunning invalid phases, but the incident is a reminder that zero scores from infrastructure failures must be excluded or repaired rather than interpreted as model behavior.

Conclusion

The 50-run extension preserves the practical recommendation but softens the strength of the claim. High root plus high GPT-5.5 direct-edit leaves remains the best observed mean-quality and quality-per-token configuration. Xhigh root plus xhigh leaves remains the most interesting ceiling configuration. But the experiment no longer supports a confident statement that high/high is statistically superior to every other top cell.

The counter-intuitive lesson is that reasoning effort is not a scalar quality knob in staged coding systems. Once multiple direct-edit leaves are involved, more reasoning can create more search, more code, and more integration burden. The root's job is not just to collect clever patches; it has to compress them into one consistent implementation. On this benchmark, the cheaper high/high configuration found the best average balance.